

✓ Sanskrit LLM Fine-Tuning — End-to-End Notebook

Single-notebook version of the project (data prep → tokenizer inspection → QLoRA fine-tuning → inference → evaluation). Originally split across `scripts/prepare_data.py`, `scripts/inspect_tokenizer.py`, `scripts/train.py`, `scripts/infer.py`, and `eval/evaluate.py`; consolidated here so the whole pipeline can run top-to-bottom in one Colab session.

Run order: Setup → Config → Data Prep → Tokenizer Inspection → Training → Inference → Evaluation. Each section is self-contained but depends on artifacts (files/variables) produced by the previous one.

✓ 1. Setup — install dependencies

```
!pip install -q -U transformers accelerate peft trl bitsandbytes datasets sacrebleu pandas
```

```
import json
import os
import random
import argparse
from pathlib import Path
from collections import defaultdict

import torch
from datasets import load_dataset
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    BitsAndBytesConfig,
)
from peft import LoraConfig, PeftModel, prepare_model_for_kbit_training
from trl import SFTTrainer, SFTConfig
import sacrebleu

random.seed(42)
```

✓ 2. Config

All the knobs that used to be CLI flags across the different scripts now live here as a single config dict, so you only need to edit one place.

```
from huggingface_hub import login
login()
```



To log in, open this URL and enter the code:

<https://hf.co/oauth/device>

A D N B - 9 K S O

Waiting for authorization...

Login successful. Logged in as **yanil-03** (token: oauth-yanil-03).

This token will be refreshed automatically when it expires.

```
CONFIG = {
    # --- data prep ---
    "max_pairs": 6000,
    "val_frac": 0.05,
    "test_frac": 0.05,
    "data_dir": "data",
    "itihasa_local_path": None, # set to a local parquet/json/jsonl path if HF auto-download fails

    # --- model / training ---
```

```

"base_model": "meta-llama/Llama-3.2-1B-Instruct", # final pick; swap to "Qwen/Qwen2.5-1.5B-Instruct" for the ungated alter
"output_dir": "outputs/lora-sanskrit",
"epochs": 2.0,
"lr": 2e-4,
"per_device_batch_size": 4,
"grad_accum": 4,
"max_seq_len": 512,
"lora_r": 16,
"lora_alpha": 32,
"lora_dropout": 0.05,
"max_train_samples": 500, # cap for a quick smoke-test run; set None for full run

# --- inference / eval ---
"max_new_tokens": 200,
"eval_limit": 100, # cap number of test examples run through inference, None = all
}

Path(CONFIG["data_dir"]).mkdir(parents=True, exist_ok=True)
Path(CONFIG["output_dir"]).mkdir(parents=True, exist_ok=True)
Path("eval").mkdir(parents=True, exist_ok=True)
CONFIG

```

```

{'max_pairs': 6000,
 'val_frac': 0.05,
 'test_frac': 0.05,
 'data_dir': 'data',
 'itihasa_local_path': None,
 'base_model': 'meta-llama/Llama-3.2-1B-Instruct',
 'output_dir': 'outputs/lora-sanskrit',
 'epochs': 2.0,
 'lr': 0.0002,
 'per_device_batch_size': 4,
 'grad_accum': 4,
 'max_seq_len': 512,
 'lora_r': 16,
 'lora_alpha': 32,
 'lora_dropout': 0.05,
 'max_train_samples': 500,
 'max_new_tokens': 200,
 'eval_limit': 100}

```

3. Data Preparation

Builds an instruction-tuning dataset for Sanskrit↔English tasks from the `itihasa` parallel corpus (with backup sources and a curated seed set as fallbacks), then expands each parallel pair into multiple instruction task types (translation both directions, explanation, QA, summarization) so the model learns instruction-following, not just one task shape.

See `REPORT.md` section 2 for full reasoning on these design choices.

```

SA2EN_TEMPLATES = [
    "Translate the following Sanskrit text into English.",
    "Provide an English translation of this Sanskrit sentence.",
    "What does this Sanskrit passage mean in English?",
]

EN2SA_TEMPLATES = [
    "Translate the following English text into Sanskrit.",
    "Provide a Sanskrit translation of this English sentence.",
    "Render this English sentence in Sanskrit.",
]

EXPLAIN_TEMPLATES = [
    "Explain the meaning and context of this Sanskrit verse in simple English.",
    "Give a brief explanation of what this Sanskrit passage is conveying.",
]

QA_TEMPLATES = [
    "Based on the Sanskrit verse below, answer the question in English.\nQuestion: What is the verse saying, in one sentence?",
    "Read the Sanskrit verse and answer: What is the main idea expressed here?",
]

SUMMARY_TEMPLATES = [
    "Summarize the following Sanskrit passage in one or two English sentences.",
]

```

```

def build_examples_from_pair(sa: str, en: str, idx: int):
    """Turn one (Sanskrit, English) pair into several instruction examples."""
    examples = []

    examples.append({
        "instruction": random.choice(SA2EN_TEMPLATES),
        "input": sa,
        "output": en,
        "task_type": "sa2en",
    })

    examples.append({
        "instruction": random.choice(EN2SA_TEMPLATES),
        "input": en,
        "output": sa,
        "task_type": "en2sa",
    })

    # Explanation task re-uses the English side as the "explanation" target.
    # Simplifying assumption (documented in REPORT.md): true explanations
    # would need a separate commentary corpus, which is scarce/inconsistent
    # in license and quality.
    if idx % 3 == 0:
        examples.append({
            "instruction": random.choice(EXPLAIN_TEMPLATES),
            "input": sa,
            "output": f"This verse conveys: {en}",
            "task_type": "explain",
        })

    if idx % 4 == 0:
        examples.append({
            "instruction": random.choice(QA_TEMPLATES),
            "input": sa,
            "output": en,
            "task_type": "qa",
        })

    if idx % 5 == 0 and len(en.split()) > 6:
        examples.append({
            "instruction": random.choice(SUMMARY_TEMPLATES),
            "input": sa,
            "output": en,
            "task_type": "summary",
        })

    return examples

```

```

def _extract_pairs_from_rows(rows, max_pairs, pairs):
    for row in rows:
        # itihasa schema: {"translation": {"sn": "...", "en": "..."}}
        tr = row.get("translation", row)
        sa = (tr.get("sn") or tr.get("sa") or "").strip()
        en = (tr.get("en") or "").strip()
        if sa and en and len(sa) > 3 and len(en) > 3:
            pairs.append((sa, en))
        if len(pairs) >= max_pairs:
            break
    return pairs

def load_itihasa(max_pairs: int):
    """Load the itihasa Sanskrit-English parallel corpus from Hugging Face.

    NOTE: HF deprecated dataset *loading scripts* (the old itihasa.py path).
    We work around this by loading the dataset's auto-generated parquet
    files directly via the `refs/convert/parquet` branch HF maintains for
    every dataset. Falls back to an empty list so the pipeline still runs
    end-to-end on the bundled seed set alone rather than crashing.
    """
    pairs = []

    try:

```

```

print("Loading rahular/itihasa via parquet (refs/convert/parquet) ...")
ds = load_dataset(
    "parquet",
    data_files={
        "train": "hf://datasets/rahular/itihasa@~parquet/default/train/0000.parquet",
        "validation": "hf://datasets/rahular/itihasa@~parquet/default/validation/0000.parquet",
        "test": "hf://datasets/rahular/itihasa@~parquet/default/test/0000.parquet",
    },
)
for split in ds:
    pairs = _extract_pairs_from_rows(ds[split], max_pairs, pairs)
    if len(pairs) >= max_pairs:
        break
if pairs:
    print(f"Loaded {len(pairs)} pairs from itihasa (parquet).")
    return pairs
except Exception as e:
    print(f"Parquet load attempt failed ({e}); trying revision='refs/convert/parquet' ...")

try:
    ds = load_dataset("rahular/itihasa", revision="refs/convert/parquet")
    for split in ds:
        pairs = _extract_pairs_from_rows(ds[split], max_pairs, pairs)
        if len(pairs) >= max_pairs:
            break
    if pairs:
        print(f"Loaded {len(pairs)} pairs from itihasa (revision fallback).")
        return pairs
except Exception as e:
    print(f"revision='refs/convert/parquet' attempt failed ({e}).")

print("Could not load itihasa from any known path; continuing with the "
      "bundled seed set only. See REPORT.md for how this was handled.")
return pairs

def load_backup_corpus(max_pairs: int):
    """Second, independent Sanskrit-English source, tried only if itihasa
    yields too few pairs to be useful for fine-tuning."""
    pairs = []
    candidates = [
        "rahular/itihasa",
        "Sanskrit-nlp/Sanskrit_English_Parallel_Corpus",
        "ganga4364/Sanskrit-English-Sentence-Pairs",
    ]

    for name in candidates:
        try:
            print(f"Trying backup source: {name} ...")
            ds = load_dataset(name)
            for split in ds:
                for row in ds[split]:
                    sa = (row.get("sanskrit") or row.get("sn") or row.get("sa")
                          or (row.get("translation", {}) or {}).get("sn", "")).strip()
                    en = (row.get("english") or row.get("en")
                          or (row.get("translation", {}) or {}).get("en", "")).strip()
                    if sa and en and len(sa) > 3 and len(en) > 3:
                        pairs.append((sa, en))
                    if len(pairs) >= max_pairs:
                        break
                if len(pairs) >= max_pairs:
                    break
            if pairs:
                print(f"Loaded {len(pairs)} pairs from backup source {name}.")
                return pairs
        except Exception as e:
            print(f" backup source {name} failed: {e}")
            continue

    return pairs

def load_itihasa_from_file(path: str, max_pairs: int):
    """Manual fallback: load itihasa (or similarly-shaped parallel data)
    from a local file, in case automatic `datasets` loading keeps failing.
    Supports .parquet, .json, and .jsonl.
    """

```

```

import pandas as pd

pairs = []
if path.endswith(".parquet"):
    df = pd.read_parquet(path)
elif path.endswith(".jsonl"):
    df = pd.read_json(path, lines=True)
else:
    df = pd.read_json(path)

for _, row in df.iterrows():
    row = row.to_dict()
    tr = row.get("translation", row)
    if isinstance(tr, str):
        continue
    sa = (tr.get("sn") or tr.get("sa") or "").strip() if tr else ""
    en = (tr.get("en") or "").strip() if tr else ""
    if sa and en and len(sa) > 3 and len(en) > 3:
        pairs.append((sa, en))
    if len(pairs) >= max_pairs:
        break

print(f"Loaded {len(pairs)} pairs from local file {path}.")
return pairs

def load_gita_fallback():
    """Small bundled seed set (Bhagavad Gita opening verses + well-known
    subhashitas), also used as a curated, hand-verifiable slice of the
    test set."""
    seed = [
        ("धर्मक्षेत्रे कुरुक्षेत्रे समवेता युयुत्सवः। मामकाः पाण्डवाश्चैव किमकुर्वत सञ्जय॥",
        "On the field of dharma, the field of Kuru, assembled and eager to fight, what did my sons and the sons of Pandu do, C
        ("कर्मण्येवाधिकारस्ते मा फलेषु कदाचन। मा कर्मफलहेतुर्भूर्मा ते सङ्गोऽस्त्वकर्मणि॥",
        "You have a right to perform your prescribed duties, but you are not entitled to the fruits of your actions. Never con
        ("योगस्थः कुरु कर्माणि सङ्गं त्यक्त्वा धनञ्जय। सिद्ध्यसिद्ध्योः समो भूत्वा समत्वं योग उच्यते॥",
        "Perform your duty established in yoga, O Dhananjaya, abandoning attachment, and be even-minded in success and failure
        ("विद्या ददाति विनयं विनयाद्याति पात्रताम्। पात्रत्वाद्धनमाप्नोति धनाद्धर्मं ततः सुखम्॥",
        "Knowledge gives discipline, from discipline comes worthiness, from worthiness one attains wealth, from wealth comes r
        ("अहिंसा परमो धर्मः धर्मं हिंसा तथैव च। धर्मे व्यवस्थितो हिंसा तस्मादहिंसा परमो धर्मः॥",
        "Non-violence is the highest duty; yet non-violence used for the protection of duty is also duty. Duty enacted in high
        ("सत्यमेव जयते नानृतं सत्येन पन्था विततो देवयानः।",
        "Truth alone triumphs, not falsehood. Through truth the divine path is spread out."),
        ("वसुधैव कुटुम्बकम्।",
        "The whole world is one family."),
        ("न त्वेवाहं जातु नासं न त्वं नेमे जनाधिपाः। न चैव न भविष्यामः सर्वे वयमतः परम्॥",
        "Never was there a time when I did not exist, nor you, nor all these kings; nor in the future shall any of us cease to
        ("उद्यमेन हि सिद्ध्यन्ति कार्याणि न मनोरथैः। न हि सुप्तस्य सिंहस्य प्रविशन्ति मुखे मृगाः॥",
        "Tasks are accomplished through effort, not through mere wishes. Deer do not walk into the mouth of a sleeping lion.")
        ("यत्र नार्यस्तु पूज्यन्ते रमन्ते तत्र देवताः।",
        "Where women are honored, there the divine beings rejoice."),
    ]
    return seed

```

```

def prepare_data(config):
    out_dir = Path(config["data_dir"])
    out_dir.mkdir(parents=True, exist_ok=True)

    pairs = []

    if config.get("itihasa_local_path"):
        try:
            pairs = load_itihasa_from_file(config["itihasa_local_path"], config["max_pairs"])
        except Exception as e:
            print(f"Could not load local file {config['itihasa_local_path']} ({e}); "
                  f"trying automatic download instead.")

    if not pairs:
        try:
            pairs = load_itihasa(config["max_pairs"])
        except Exception as e:
            print(f"Could not load itihasa ({e}); trying backup sources.")
            pairs = []

    MIN_USABLE_PAIRS = 200

```

```

if len(pairs) < MIN_USABLE_PAIRS:
    print(f"Only {len(pairs)} pairs from itihasa (need >= {MIN_USABLE_PAIRS}); "
          f"trying backup corpora ...")
    try:
        backup_pairs = load_backup_corpus(config["max_pairs"])
        pairs.extend(backup_pairs)
    except Exception as e:
        print(f"Backup corpus loading also failed: {e}")

if len(pairs) < MIN_USABLE_PAIRS:
    print(
        f"\n*** WARNING: only {len(pairs)} real parallel pairs available "
        f"(plus the 10-verse curated seed set). This is too little data "
        f"for meaningful fine-tuning. ***\n"
        "Fix options, in order of preference:\n"
        " 1. In Colab, restart runtime and re-run this cell (transient HF/network issues are common).\n"
        " 2. pip install -U datasets\n"
        " 3. Manually download from https://huggingface.co/datasets/rahular/itihasa/tree/main "
        "and set CONFIG['itihasa_local_path'].\n"
        " 4. Proceed with the small seed set for a PIPELINE SMOKE TEST only.\n"
    )

seed_pairs = load_gita_fallback()

all_pairs = pairs
random.shuffle(all_pairs)

all_examples = []
for i, (sa, en) in enumerate(all_pairs):
    all_examples.extend(build_examples_from_pair(sa, en, i))

random.shuffle(all_examples)

n = len(all_examples)
n_val = int(n * config["val_frac"])
n_test = int(n * config["test_frac"])

val = all_examples[:n_val]
test = all_examples[n_val:n_val + n_test]
train = all_examples[n_val + n_test:]

seed_examples = []
for i, (sa, en) in enumerate(seed_pairs):
    seed_examples.extend(build_examples_from_pair(sa, en, i))
test.extend(seed_examples)

def dump(rows, path):
    with open(path, "w", encoding="utf-8") as f:
        for r in rows:
            f.write(json.dumps(r, ensure_ascii=False) + "\n")

dump(train, out_dir / "train.jsonl")
dump(val, out_dir / "val.jsonl")
dump(test, out_dir / "test.jsonl")

print(f"train={len(train)} val={len(val)} test={len(test)}")
print(f"Written to {out_dir}/")

prepare_data(CONFIG)

```

```

Loading rahular/itihasa via parquet (refs/convert/parquet) ...
Parquet load attempt failed (Unable to find 'hf://datasets/rahular/itihasa@~parquet/default/train/0000.parquet'); trying revision
Itihasa/train/0000.parquet: reconstructing file: 100% 16.4MB / 16.4MB, 1.53MB/s

Itihasa/train/0000.parquet: downloading bytes: 16.3MB, 1.48MB/s

Itihasa/validation/0000.parquet: reconstructing file: 100% 1.39MB / 1.39MB, 138kB/s

Itihasa/validation/0000.parquet: downloading bytes: 1.39MB, 138kB/s

Itihasa/test/0000.parquet: reconstructing file: 100% 2.61MB / 2.61MB, 256kB/s

Itihasa/test/0000.parquet: downloading bytes: 2.61MB, 255kB/s

Generating train split: 75162/0 [00:00<00:00, 326515.09 examples/s]

Generating validation split: 6149/0 [00:00<00:00, 186680.00 examples/s]

Generating test split: 11722/0 [00:00<00:00, 214714.90 examples/s]

Loaded 6000 pairs from itihasa (revision fallback).
train=15030 val=835 test=864
Written to data/

```

4. Tokenizer Inspection

Quick diagnostic: how badly does the base model's tokenizer fragment Sanskrit (Devanagari) text compared to English text of similar length? Directly addresses the "Tokenization / Vocabulary" challenge in the assignment brief, and is useful evidence when justifying a base-model choice in the report.

```

TOKENIZER_SAMPLES = [
    ("वसुधैव कुटुम्बकम्!", "The whole world is one family."),
    ("धर्मक्षेत्रे कुरुक्षेत्रे समवेता युयुत्सवः।",
     "On the field of dharma, the field of Kuru, assembled and eager to fight."),
    ("कर्मण्येवाधिकारस्ते मा फलेषु कदाचन।",
     "You have a right to perform your duty, but not to the fruits of action."),
    ("सत्यमेव जयते।", "Truth alone triumphs."),
    ("अहिंसा परमो धर्मः।", "Non-violence is the highest duty."),
]

def inspect_tokenizer(model_name):
    tok = AutoTokenizer.from_pretrained(model_name)

    print(f"Tokenizer: {model_name}")
    print(f"Vocab size: {tok.vocab_size}\n")

    report = []
    for sa, en in TOKENIZER_SAMPLES:
        sa_ids = tok.encode(sa, add_special_tokens=False)
        en_ids = tok.encode(en, add_special_tokens=False)

        sa_tpc = len(sa_ids) / max(len(sa), 1)
        en_tpc = len(en_ids) / max(len(en), 1)

        print(f"SA: {sa} chars={len(sa)} tokens={len(sa_ids)} tokens/char={sa_tpc:.2f}")
        print(f"EN: {en} chars={len(en)} tokens={len(en_ids)} tokens/char={en_tpc:.2f}")
        print(f"--> Sanskrit costs {sa_tpc / en_tpc:.2f}x more tokens/char than English\n")

        report.append({
            "sanskrit": sa, "english": en,
            "sa_chars": len(sa), "sa_tokens": len(sa_ids), "sa_tokens_per_char": round(sa_tpc, 3),
            "en_chars": len(en), "en_tokens": len(en_ids), "en_tokens_per_char": round(en_tpc, 3),
            "ratio_sa_over_en": round(sa_tpc / en_tpc, 2),
        })

    avg_ratio = sum(r["ratio_sa_over_en"] for r in report) / len(report)
    print(f"Average Sanskrit/English tokens-per-char ratio: {avg_ratio:.2f}x")
    return {"model": model_name, "samples": report, "avg_ratio": avg_ratio}

# Compare the candidate base model(s). Add/remove entries as needed.
tokenizer_reports = {}
for m in [CONFIG["base_model"]]:
    tokenizer_reports[m] = inspect_tokenizer(m)

```

```
with open("eval/tokenizer_report.json", "w", encoding="utf-8") as f:
    json.dump(tokenizer_reports, f, ensure_ascii=False, indent=2)
```

```
config.json: 100%                               877/877 [00:00<00:00, 81.9kB/s]
tokenizer_config.json: 100%                     54.5k/54.5k [00:00<00:00, 4.94MB/s]
tokenizer.json: 100%                             9.09M/9.09M [00:00<00:00, 41.1MB/s]
special_tokens_map.json: 100%                   296/296 [00:00<00:00, 17.7kB/s]
Tokenizer: meta-llama/Llama-3.2-1B-Instruct
Vocab size: 128000
```

SA: वसुधैव कुटुम्बकम्। chars=18 tokens=13 tokens/char=0.72
 EN: The whole world is one family. chars=30 tokens=7 tokens/char=0.23
 --> Sanskrit costs 3.10x more tokens/char than English

SA: धर्मक्षेत्रे कुरुक्षेत्रे समवेता युयुत्सवः। chars=43 tokens=25 tokens/char=0.58
 EN: On the field of dharma, the field of Kuru, assembled and eager to fight. chars=72 tokens=19 tokens/char=0.26
 --> Sanskrit costs 2.20x more tokens/char than English

SA: कर्मण्येवाधिकारस्ते मा फलेषु कदाचन। chars=35 tokens=21 tokens/char=0.60
 EN: You have a right to perform your duty, but not to the fruits of action. chars=71 tokens=17 tokens/char=0.24
 --> Sanskrit costs 2.51x more tokens/char than English

SA: सत्यमेव जयते। chars=13 tokens=6 tokens/char=0.46
 EN: Truth alone triumphs. chars=21 tokens=5 tokens/char=0.24
 --> Sanskrit costs 1.94x more tokens/char than English

SA: अहिंसा परमो धर्मः। chars=18 tokens=11 tokens/char=0.61
 EN: Non-violence is the highest duty. chars=33 tokens=8 tokens/char=0.24
 --> Sanskrit costs 2.52x more tokens/char than English

Average Sanskrit/English tokens-per-char ratio: 2.45x

5. Training (QLoRA fine-tuning)

4-bit QLoRA fine-tuning of the base model on the Sanskrit↔English instruction dataset built above. Designed to run on a single free-tier Colab GPU (T4 16GB or L4).

Model choice: default is `meta-llama/Llama-3.2-1B-Instruct`. Early on this looked like the wrong call — it trained ~~10x slower than Qwen/Qwen2.5-1.5B-Instruct~~ on what I thought was an identical config (6 hrs vs ~20 min for 500 samples). Once I added the device-map/attention diagnostics right after model load (see the print statements below), that turned out to be an environment-specific bottleneck rather than anything inherent to the model — with it resolved, Llama trains in ~6 minutes for 500 samples versus Qwen's ~22 minutes on the same config, and it also fragments Sanskrit less (2.45x tokens/char vs Qwen's 4.02x). Both facts now favor Llama, which is why it's the default here. `Qwen/Qwen2.5-1.5B-Instruct` is still wired in as an ungated fallback if you don't want to deal with Llama's license-acceptance step.

```
print(f"Loading tokenizer/model: {CONFIG['base_model']}")
tokenizer = AutoTokenizer.from_pretrained(CONFIG["base_model"])
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=True,
)

model = AutoModelForCausalLM.from_pretrained(
    CONFIG["base_model"],
    quantization_config=bnb_config,
    device_map="auto",
)

# Diagnostics: confirm no CPU offload and check attention backend, since both
# can cause dramatic (10-20x) slowdowns on memory-constrained GPUs like a T4.
# `hf_device_map` only exists when accelerate actually had to split the model
# across devices; if it's absent, the whole model fit on one device (good).
device_map = getattr(model, "hf_device_map", None)
if device_map is not None:
    print("Device map:", device_map)
```

```

    if any(str(d) == "cpu" for d in device_map.values()):
        print("!! WARNING: some layers offloaded to CPU -- this will be very slow.")
    else:
        print("Device map: not set by accelerate -> model loaded fully on a single device:",
              next(model.parameters()).device)
    print("Attention implementation:", model.config._attn_implementation)

    model = prepare_model_for_kbit_training(model)

```

```

Loading tokenizer/model: meta-llama/Llama-3.2-1B-Instruct
model.safetensors: reconstructing file: 100%                2.47GB / 2.47GB, 105MB/s

model.safetensors: downloading bytes:                      2.14GB, 96.7MB/s

Loading weights: 100%                146/146 [00:04<00:00, 49.67it/s]

generation_config.json: 100%                189/189 [00:00<00:00, 19.1kB/s]

Device map: not set by accelerate -> model loaded fully on a single device: cuda:0
Attention implementation: sdpa

```

```

def format_example(example, tokenizer):
    """Build a single training string using the model's chat template if it
    has one (instruct models), else fall back to a raw Alpaca-style template."""
    instruction = example["instruction"]
    inp = example.get("input", "")
    output = example["output"]

    if getattr(tokenizer, "chat_template", None):
        messages = [
            {"role": "user", "content": f"{instruction}\n\n{inp}".strip()},
            {"role": "assistant", "content": output},
        ]
        text = tokenizer.apply_chat_template(messages, tokenize=False)
    else:
        text = (
            f"### Instruction:\n{instruction}\n\n"
            f"### Input:\n{inp}\n\n"
            f"### Response:\n{output}"
        )
    return {"text": text}

```

```

lora_config = LoraConfig(
    r=CONFIG["lora_r"],
    lora_alpha=CONFIG["lora_alpha"],
    lora_dropout=CONFIG["lora_dropout"],
    bias="none",
    task_type="CAUSAL_LM",
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
                    "gate_proj", "up_proj", "down_proj"],
)

print("Loading datasets ...")
train_ds = load_dataset("json", data_files=f"{CONFIG['data_dir']}/train.jsonl", split="train")
val_ds = load_dataset("json", data_files=f"{CONFIG['data_dir']}/val.jsonl", split="train")

if CONFIG["max_train_samples"]:
    train_ds = train_ds.select(range(min(CONFIG["max_train_samples"], len(train_ds))))

train_ds = train_ds.map(lambda ex: format_example(ex, tokenizer))
val_ds = val_ds.map(lambda ex: format_example(ex, tokenizer))

sft_config = SFTConfig(
    output_dir=CONFIG["output_dir"],
    num_train_epochs=CONFIG["epochs"],
    per_device_train_batch_size=CONFIG["per_device_batch_size"],
    per_device_eval_batch_size=CONFIG["per_device_batch_size"],
    gradient_accumulation_steps=CONFIG["grad_accum"],
    learning_rate=CONFIG["lr"],
    logging_steps=20,
    eval_strategy="steps",
    eval_steps=100,
    save_strategy="steps",
    save_steps=200,
    save_total_limit=2,
)

```

```

        bf16=True,
        max_length=CONFIG["max_seq_len"],
        dataset_text_field="text",
        report_to="none",
        warmup_steps=56,
        lr_scheduler_type="cosine",
        gradient_checkpointing=True,
        optim="paged_adamw_8bit",
    )

    trainer = SFTTrainer(
        model=model,
        args=sft_config,
        train_dataset=train_ds,
        eval_dataset=val_ds,
        peft_config=lora_config,
    )

```

Loading datasets ...

Generating train split: 15030/0 [00:00<00:00, 231295.89 examples/s]

Generating train split: 835/0 [00:00<00:00, 27727.15 examples/s]

Map: 100% 500/500 [00:00<00:00, 4355.88 examples/s]

Map: 100% 835/835 [00:00<00:00, 6977.89 examples/s]

Adding EOS to train dataset: 100% 500/500 [00:00<00:00, 8966.98 examples/s]

Tokenizing train dataset: 100% 500/500 [00:00<00:00, 1522.47 examples/s]

Building labels for train dataset: 100% 500/500 [00:00<00:00, 3897.39 examples/s]

Truncating train dataset: 100% 500/500 [00:00<00:00, 4195.49 examples/s]

Dropping fully masked examples from train dataset: 100% 500/500 [00:00<00:00, 5521.95 examples/s]

Adding EOS to eval dataset: 100% 835/835 [00:00<00:00, 10601.55 examples/s]

Tokenizing eval dataset: 100% 835/835 [00:00<00:00, 1587.88 examples/s]

Building labels for eval dataset: 100% 835/835 [00:00<00:00, 4934.33 examples/s]

Truncating eval dataset: 100% 835/835 [00:00<00:00, 3486.30 examples/s]

Dropping fully masked examples from eval dataset: 100% 835/835 [00:00<00:00, 6414.53 examples/s]

```

print("Starting training ...")
trainer.train()

```

```

print(f"Saving adapter to {CONFIG['output_dir']}")
trainer.save_model(CONFIG["output_dir"])
tokenizer.save_pretrained(CONFIG["output_dir"])

```

```

with open(os.path.join(CONFIG["output_dir"], "run_config.json"), "w") as f:
    json.dump(CONFIG, f, indent=2, default=str)

```

```

print("Done training.")

```

[transformers] The tokenizer has new PAD/BOS/EOS tokens that differ from the model config and generation config. The model config is outdated. Starting training ...

[64/64 17:25, Epoch 2/2]

Step	Training Loss	Validation Loss	Entropy	Num Tokens	Mean Token Accuracy
64	2.400049	2.315050	2.349290	151606.000000	0.542706

```

Saving adapter to outputs/lora-sanskrit
Done training.

```

6. Inference

Load the base model with (optionally) the fine-tuned LoRA adapter and run inference — either a single ad-hoc prompt, or batch inference over the test set (once for the base model to get "before" outputs, once with the adapter for "after" outputs, so `evaluate.py`'s before/after table has both).

```

def load_inference_model(base_model, adapter_path=None):
    tok = AutoTokenizer.from_pretrained(adapter_path if adapter_path else base_model)
    if tok.pad_token is None:
        tok.pad_token = tok.eos_token

    bnb_config = BitsAndBytesConfig(
        load_in_4bit=True,
        bnb_4bit_quant_type="nf4",
        bnb_4bit_compute_dtype=torch.bfloat16,
        bnb_4bit_use_double_quant=True,
    )

    m = AutoModelForCausalLM.from_pretrained(
        base_model,
        quantization_config=bnb_config,
        device_map="auto",
    )

    if adapter_path:
        m = PeftModel.from_pretrained(m, adapter_path)

    m.eval()
    return m, tok

def generate(m, tok, instruction, input_text, max_new_tokens=200):
    if getattr(tok, "chat_template", None):
        messages = [{"role": "user", "content": f"{instruction}\n\n{input_text}".strip()}]
        prompt = tok.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)
    else:
        prompt = f"### Instruction:\n{instruction}\n\n### Input:\n{input_text}\n\n### Response:\n"

    inputs = tok(prompt, return_tensors="pt").to(m.device)
    with torch.no_grad():
        out = m.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False,
            temperature=None,
            top_p=None,
            pad_token_id=tok.pad_token_id,
        )
    text = tok.decode(out[0][inputs["input_ids"].shape[1]:], skip_special_tokens=True)
    return text.strip()

def run_batch_inference(base_model, adapter_path, test_file, out_file, max_new_tokens=200, limit=None):
    m, tok = load_inference_model(base_model, adapter_path)

    with open(test_file, encoding="utf-8") as f:
        lines = f.readlines()
    if limit:
        lines = lines[:limit]

    results = []
    for i, line in enumerate(lines):
        ex = json.loads(line)
        pred = generate(m, tok, ex["instruction"], ex.get("input", ""), max_new_tokens)
        results.append({
            "instruction": ex["instruction"],
            "input": ex.get("input", ""),
            "reference": ex["output"],
            "prediction": pred,
            "task_type": ex.get("task_type", "unknown"),
        })
    if (i + 1) % 10 == 0:
        print(f"{i + 1}/{len(lines)} done")

    with open(out_file, "w", encoding="utf-8") as f:
        for r in results:
            f.write(json.dumps(r, ensure_ascii=False) + "\n")
    print(f"Wrote {len(results)} predictions to {out_file}")

    # free GPU memory before the next model load
    del m
    torch.cuda.empty_cache()

```

```
return results
```

```
# "Before" - base model only, no adapter
# CONFIG["eval_limit"]=100
run_batch_inference(
    base_model=CONFIG["base_model"],
    adapter_path=None,
    test_file=f"{CONFIG['data_dir']}/test.jsonl",
    out_file="eval/predictions_base.jsonl",
    max_new_tokens=CONFIG["max_new_tokens"],
    limit=CONFIG["eval_limit"],
)
```


[illegible]

the shining and radiant sun, the beautiful and divine moon, the shining and radiant sun, the beautiful and divine moon, the shining and radiant sun, the beautiful and divine moon, the shining and radiant sun, the beautiful and divine moon, the shining and radiant sun, the beautiful and divine moon, the shining and radiant sun, the beautiful and divine moon, the shining and radiant sun, the beautiful and

```
{
    'task_type': 'sa2en'},
    {'instruction': 'Translate the following Sanskrit text into English.',
     'input': 'सुकुमारं च बालं च सततं ललितं त्वया। क तात भरतं हित्वा विलपन्तं गतो भवान् ॥',
     'reference': 'O father, where have you gone, leaving the tender and youthful Bharata fondled by you, to lament (your loss).',
     'prediction': 'I can provide the translation of the Sanskrit text. Here it is:\n\n"O my beloved, my son, my eternal love, I am always devoted to you. I am always in love with you, and I am always devoted to you. I am always in love with you, and I am always devoted to you."'\n\nThis is a poem from the Bhagavad Gita, a Hindu scripture.'},
    'task_type': 'sa2en'},
    {'instruction': 'Render this English sentence in Sanskrit.',
     'input': 'I deprecate your displeasure. O Kausalyā with joined hands. You are ever affectionate and did not treat harshly even enemies.',
     'reference': 'प्रसादये त्वां कौसल्ये रचितोऽयं मयाञ्जलिः। वत्सला चानृशंसा च त्वं हि नित्यं परेष्वपि॥',
     'prediction': 'The Sanskrit translation of the given English sentence is:\n\nमै तुम्हारे दुःप्रिय को देखकर विनम्र हूँ। कौसल्ये (Kausalya) के साथ हाथ मिलाकर। तुम्हारे कृपालु (Krupal) ने किसी भी दुश्मन को कठोर रूप से नहीं देखा।',
     'task_type': 'en2sa'}},
    {'instruction': 'Translate the following English text into Sanskrit.',
     'input': "Hearing those words of the intelligent prince, that righteous (ascetic) brought a bull* as well as Arghya and water and divers kinds of edibles consisting of wild fruits and roots. *When an eminent guest came to a person's house, as he offered madhuharka, with a bull which was killed for his entertainment",
     'reference': 'तस्य तद् वचनं श्रुत्वा राजपुत्रस्य धीमतः। उपानयत धर्मात्मा गामर्थमुदकं ततः॥',
     'prediction': 'हे विद्वान् प्रजा के शब्दों के साथ, ज्ञानी राजा ने एक बुल के साथ, अर्य और पानी और विभिन्न प्रकार के भोजन के साथ, जंगली फलों और पौधों के विभिन्न प्रकार के भोजन का सेवन किया। *जब एक महान मेहमान अपने घर के पास आया, तो वह माधुपर्का के साथ, जो कि उसके मनोरंजन के लिए मारा गया था,',
     'task_type': 'en2sa'}},
    {'instruction': 'Translate the following English text into Sanskrit.',
     'input': 'Khara stationed on his car in the midst of Rākṣasas appeared like red bodied one risen in the midst of the stars.',
     'reference': 'स तेषां यातुधानानां मध्ये रथगतः खरः। बभूव मध्ये ताराणां लोहिताङ्ग इवोद्धतः॥',
     'prediction': 'Here\'s the translation of the given English text into Sanskrit:\n\n"Khara, the charioteer, was seated in his chariot in the midst of the stars, and the Rākṣasas, the celestial beings, rose up from the stars."'\n\nNote: In Sanskrit, the word "Rākṣasas" is derived from the root "Rakṣa" meaning "serpent" or "serpent-like being", and "Asa" meaning "star".',
     'task_type': 'en2sa'}},
    {'instruction': 'Explain the meaning and context of this Sanskrit verse in simple English.',
     'input': 'सर्वतश्चार्यतां दृष्टिः कानने निपुणो ह्रासि । आश्रमः कतरस्मिन् नो देशे भवति सम्मतः॥',
     'reference': 'This verse conveys: Do you cast your eyes around this forest, and (ascertain) what spot shall suit our asylum.',
     'prediction': "This is a Sanskrit verse from the Bhagavad Gita, a Hindu scripture. Here's a simplified explanation of its meaning and context:\n\nThe verse is saying that when people are free from worldly desires (सर्वतश्चार्यतां) and are fully awake (दृष्टिः), they will be able to attain spiritual enlightenment (कानने निपुणो ह्रासि) and will be able to see the world (आश्रमः कतरस्मिन्) no matter where they are (सम्मतः).\n\nIn simpler terms, it means that when people are free from worldly attachments and are fully aware of their spiritual nature, they will be able to see the world in its true form and understand the nature of reality (सम्मतः).",
     'task_type': 'explain'}},
    {'instruction': 'Translate the following Sanskrit text into English.',
     'input': 'सा मानुमन्यस्व वनं व्रजन्तं कुरुष्व नः स्वस्त्ययनानि देवि। यथा समाप्ते पुनराव्रजेयं यथा हि सत्येन पुनर्यायतिः॥',
     'reference': 'Do you permit me therefore, to repair to the forest and perform benedictory ceremonies for me so that I may again return home like to Yayāti regaining heaven by truth.',
     'prediction': 'I can provide the translation of the Sanskrit text. Here it is:\n\n"मानुमन्यस्व वनं व्रजन्तं कुरुष्व नः स्वस्त्ययनानि देवि। यथा समाप्ते पुनराव्रजेयं यथा हि सत्येन पुनर्यायतिः"\n\nThis is a Vedic text, which is a collection of ancient Indian scriptures. Here\'s a breakdown of the translation:\n\n.\n*\n"मानुमन्यस्व" (Manumnyas) means "I am a student" or "I am a seeker of truth".\n*\n"वनं" (Vana) means "mountain" or "forest".\n*\n"व्रजन्तं" (Vrjantha) means "I am a hunter" or "I am a seeker of truth".\n*\n"कुरुष्व" (Kurusva) means "mountain",
     'task_type': 'sa2en'},
    {'instruction': 'Translate the following English text into Sanskrit.',
     'input': 'Her neck had the hue of sandal and was adorned with necklace—that beautiful tender neck was eaten up by the Rākṣasas, my beloved wife wailing.',
     'reference': 'सा हि चन्दनकवचाभा ग्रीवा ग्रैवेयकोचिता। कोमला विलपन्त्यास्तु कान्ताया भक्षिता शुभा॥',
     'prediction': 'मेरी नाक का रंग सैंडल के था, और वह मेरे प्रिय पति के साथ मेरे प्रेमी था। वह मुझे खा गया।',
     'task_type': 'en2sa'}},
    {'instruction': 'Provide a Sanskrit translation of this English sentence.',
     'input': 'You shall regain your stalwart and beautiful shape when you shall be burnt by Rāma in a dense forest having got your arms dismembered by him.',
     'reference': 'यदा छित्वा भुजौ रामस्तां दहेद् विजने वने॥ तदा त्वं प्राप्स्यसे रूपं स्वमेव विपुलं शुभम्। श्रिया विराजितं पुत्रं दनोस्त्वं विद्धि लक्ष्मणा॥',
     'prediction': "Here\'s the Sanskrit translation of the given English sentence:\n\nतुम्हें राम के द्वारा मिट्टी के मैदान में घूमने पर अपने साहसिक और सुंदर रूप को पुनः प्राप्त होने के बाद होगा॥",
     'task_type': 'en2sa'}},
    {'instruction': 'Render this English sentence in Sanskrit.',
     'input': 'Thus addressed by the monarch, Rāma, the best of the righteous, well versed in speech, with joined hands addressed his father thus.',
     'reference': 'एवमुक्तो नृपतिनां रामो धर्मभूतां वरः। प्रत्युवाचाञ्जलिं कृत्वा पितरं वाक्यकोविदः॥',
     'prediction': 'Here is the English sentence rendered in Sanskrit:\n\nरामा राजा ने स्वस्थ वचन, संगत होने वाले सर्वश्रेष्ठ न्यायसंगम, साथ में हाथों में अपने पिता को प्रार्थना किया।',
     'task_type': 'en2sa'}},
    {'instruction': 'Read the Sanskrit verse and answer: What is the main idea expressed here?',
     'input': 'त्वमशोकस्य शाखाभिः पुष्पप्रियतरा प्रिये। आवृणोषि शरीरं ते मम शोकविवर्धनी॥',
     'reference': 'O my dear, you delights greatly in flowers. Covering your own Person with Asoka twigs you are increasing my grief.',
     'prediction': "The main idea expressed in this Sanskrit verse is the importance of the soul (प्रिय) and the fleeting nature of life (पुष्पप्रियतरा). The verse is a prayer to the gods, asking for the soul's protection and the preservation of life.",
     'task_type': 'qa'}}
```